

Dipartimento di Elettronica, Informazione e
Bioingegneria



**POLITECNICO
DI MILANO**

Corso di Laurea Triennale in Ingegneria Informatica

PROVA FINALE RETI LOGICHE

Docente:
Prof. Gianluca Palermo

Relazione di:
Federico Quartieri

Anno accademico 2023/2024

Indice

1 Introduzione

2 Architettura

2.1	Modulo 1	
2.2	Modulo 2	
2.3	Modulo 3	
2.4	Modulo 4	
2.5	FSM	

3 Risultati sperimentali

3.1	Sintesi	
3.2	Simulazioni	
3.2.1	Test bench 1	
3.2.2	Test bench 2	
3.2.3	Test bench 3	
3.2.4	Test bench 4	
3.2.5	Test bench 5	

4 Conclusioni

Introduzione

Il progetto consiste nello sviluppo di un componente hardware in VHDL che si interfacci con una memoria esterna. Dato un indirizzo di memoria iniziale e un numero k di elementi da leggere, il componente deve modificare una sequenza di interi in memoria.

Il componente riceverà in input prima un segnale di *reset* e successivamente un segnale di *start* (che indica la richiesta di codifica)

Una volta conclusa l'esecuzione il componente dovrà portare ad 1 il segnale *done*, e aspettare che il segnale *start* in input sia uguale a 0.

Quando il segnale *start* è 0, il componente dovrà portare a 0 il segnale *done*, successivamente potranno essere notificate altre richieste di codifica (segnale *start* = 1), senza che avvenga un reset.

La sequenza in memoria è nella forma seguente:

w_1	c_1	w_2	c_2	...	w_i	c_i	...	w_k	c_k
-------	-------	-------	-------	-----	-------	-------	-----	-------	-------

È richiesto leggere w_i e modificare la memoria nel modo seguente:

- $w_i \neq 0 \implies w_i = w_i \quad ; \quad c_i = 31 \quad \forall i = 1, \dots, k$
- $w_i = 0 \implies w_i = w_{i-1} \quad ; \quad c_i = c_{i-1} - 1 \quad \forall i = 1, \dots, k$

Note:

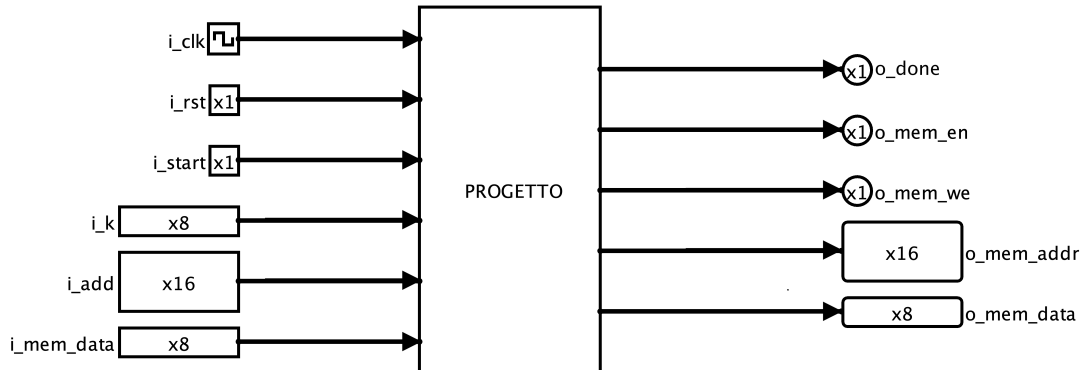
- $c \in [0, 31]$
- w_{i-1} e c_{i-1} sono intesi quelli già aggiornati dal componente

Esempio con memoria prima e dopo l'esecuzione

128	0	64	0	0	0	0	0	0	0	0	0	0	0	100	0	1	0	0	0	5	0	23	0	200	0	0	0
128	31	64	31	64	30	64	29	64	28	64	27	64	26	100	31	1	31	1	30	5	31	23	31	200	31	200	30

Architettura

L'architettura del componente sintetizzato è la seguente



Si interfaccia con un componente di memoria, tramite i seguenti segnali:

- in uscita (e in ingresso in memoria)
 - *o_mem_addr*: indirizzo da cui leggere in memoria
 - *o_mem_data*: dati da scrivere in memoria
 - *o_mem_en*: abilita memoria (se *we* = 0 la memoria è abilitata solo in lettura)
 - *o_mem_we*: abilita memoria in scrittura
- in ingresso (e in uscita dalla memoria)
 - *i_mem_data*: dati letti dalla memoria

Vengono inoltre forniti i seguenti segnali in ingresso:

- *i_k*: numero di interi da leggere dalla memoria
- *i_add*: indirizzo della memoria da cui iniziare a leggere
- *i_clk*: segnale di clock
- *i_rst*: segnale di reset che ripristina in modo asincrono tutti i registri
- *i_start*: segnale che notifica l'inizio dell'esecuzione

Infine *o_done* è il segnale che il componente dovrà portare ad 1 una volta conclusa l'esecuzione.

Il componente è diviso in 4 sottomoduli e un FSM (Finite State Machine).

Ognuno dei 4 sottomoduli ha un registro (definito in VHDL come process).

Ogni registro ha un segnale di clock e reset, comuni a tutti i registri (sono input dell'architettura generale), i registri modificano il proprio stato sul fronte di salita del clock (se hanno il segnale di load uguale ad 1).

Ogni modulo ha i propri segnali di ingresso e uscita (nei circuiti in figura mostrati in seguito sono contrassegnati da un'etichetta in maiuscolo), che possono essere i segnali sopracitati nell'architettura generale o segnali che vengono determinati dall'fsm o determinano il comportamento della stessa.

Inoltre ogni modulo ha i propri segnali interni (nei circuiti in figura in minuscolo).

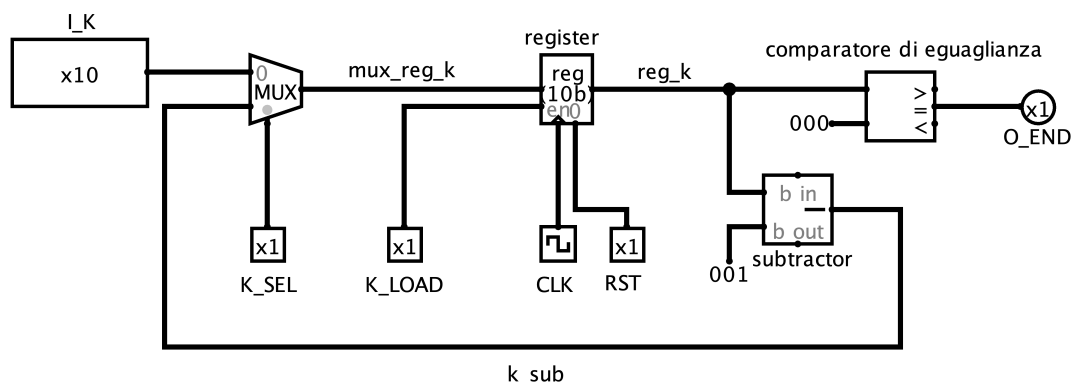
I 4 sottomoduli sono i seguenti:

- Modulo 1: aggiornamento del numero di dati da leggere in memoria (k)
- Modulo 2: aggiornamento della credibilità (c)
- Modulo 3: aggiornamento dell'indirizzo di memoria (add)
- Modulo 4: aggiornamento dell'ultimo dato letto dalla memoria (data)

Note per le figure seguenti:

- Le costanti sono rappresentate in esadecimale per facilitarne la rappresentazione, per esempio nel secondo modulo la costante da sottrarre è rappresentata con 2 cifre essendo un numero a 8 bit.
- Ogni segnale può essere di un numero diverso di bit, questa informazione è visibile nei segnali di input e di output

Modulo 1



Ingressi:

- i_k (input componente)
- k_{sel} (da fsm)
- k_{load} (da fsm): abilita scrittura nel registro

Uscite:

- o_end (per fsm): indica se $k = 0$ (esecuzione terminata)

Costanti:

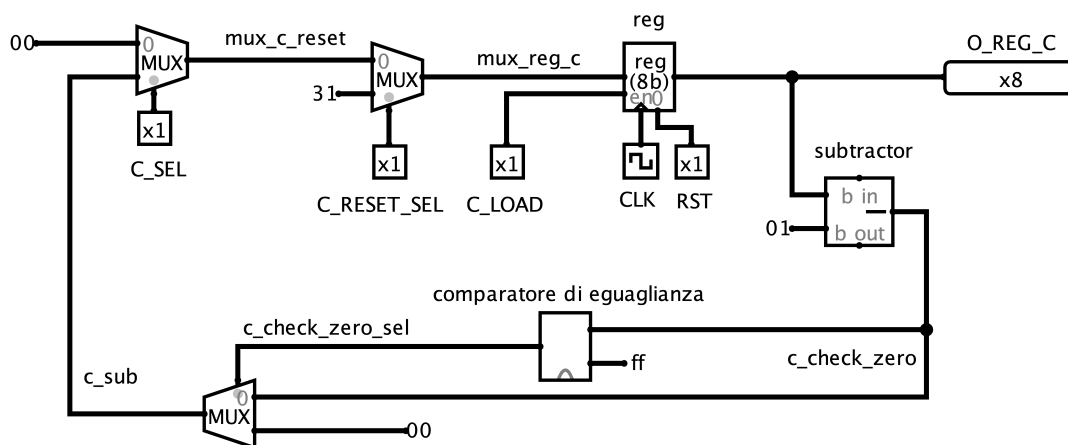
- 1: valore da sottrarre a k
- 0: valore da comparare con k

Il primo multiplexer serve a selezionare se nel registro venga salvato i_k ($k_sel = 0$) oppure il valore aggiornato di k (k sottratto di 1). All'inizio dell'esecuzione k_sel è uguale a 0 per inizializzare k ad i_k , successivamente k_sel sarà sempre uguale ad 1 per tutta la durata di una singola esecuzione.

k_load viene posto ad 1 per aggiornare il registro.

Questo modulo serve a calcolare il nuovo k , decrementandolo di 1 ad ogni lettura, e a notificare se si è raggiunti la fine di un'esecuzione, portando ad 1 il valore di o_end se $k = 0$.

Modulo 2



Ingressi:

- c_sel (da fsm)
- c_reset_sel (da fsm)
- c_load (da fsm)

Uscite:

- o_reg_c (per output componente)

Costanti:

- 0: valore da assegnare a c
 - 31: valore a cui ripristinare c
 - 1: valore da sottrarre a c
 - $255(ff_{hex})$: valore da comparare a c_check_zero
-

Il primo multiplexer serve a selezionare se nel registro venga salvato 0 ($c_sel = 0$) oppure il valore aggiornato di c (c sottratto di 1). All'inizio dell'esecuzione c_sel è uguale a 0 per inizializzare c a 0, successivamente c_sel sarà sempre uguale ad 1 per tutta la durata di una singola esecuzione.

c_load viene posto ad 1 per aggiornare il registro.

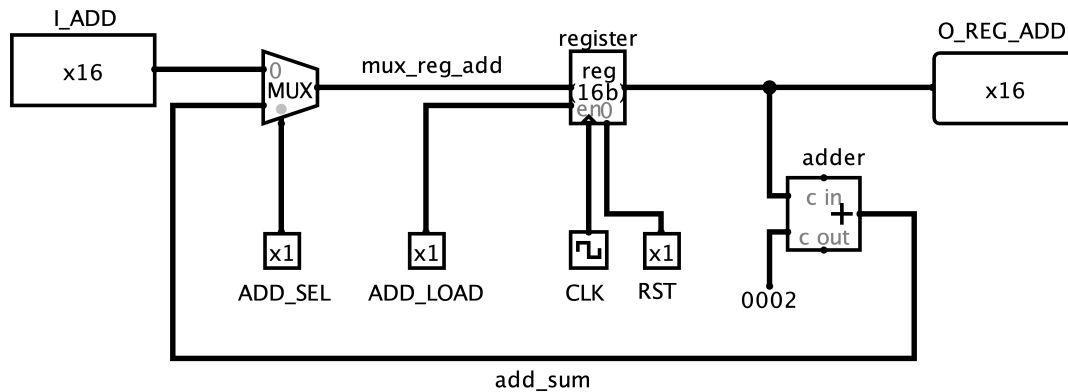
Il secondo multiplexer serve a ripristinare c a 31 se viene letto in memoria un valore diverso da zero.

Se $c = 0$ prima della sottrazione, $00000000 - 00000001 = 11111111$ (255_{dec} ; ff_{hex}).

Per questo motivo il nuovo valore di c , prima di essere riportato in ingresso al primo multiplexer, viene posto a 0 se uguale a 255.

Questo modulo serve a calcolare il nuovo c , aggiornarlo quando necessario e portarlo in uscita affinché venga scritto in memoria quando necessario.

Modulo 3



Ingressi:

- *i_add* (da input componente)
- *add_sel* (da fsm)
- *add_load* (da fsm)

Uscite:

- *o_reg_add* (per output componente)

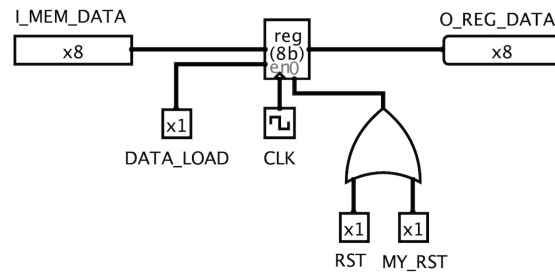
Costanti:

- 2: valore da addizionare ad add

Il primo multiplexer serve a selezionare se nel registro venga salvato *i_add* (*add_sel* = 0) oppure il valore aggiornato dell'indirizzo (add addizionato di 2). All'inizio dell'esecuzione *add_sel* è uguale a 0 per inizializzare l'indirizzo a *i_add*, successivamente *add_sel* sarà sempre uguale ad 1 per tutta la durata di una singola esecuzione. *add_load* viene posto ad 1 per aggiornare il registro.

Questo modulo serve a calcolare il nuovo indirizzo da cui leggere in memoria, incrementandolo di 2 ad ogni lettura, e portarlo in uscita affinché venga utilizzato per definire l'indirizzo su cui scrivere in memoria quando necessario

Modulo 4



Ingressi:

- `i_mem_data` (da input componente)

Uscite:

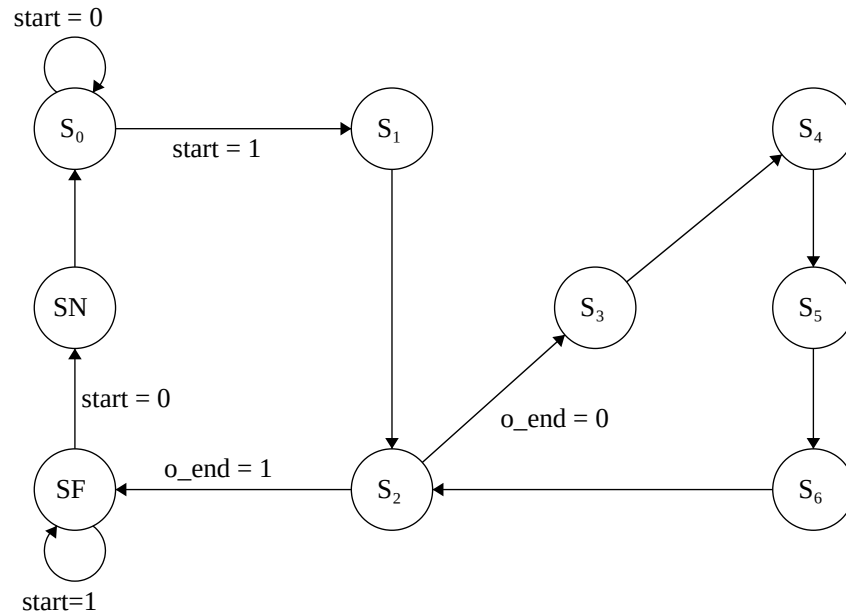
- `o_reg_data` (per output componente)

`data_load` viene posto ad 1 per aggiornare il registro.

Questo modulo serve ad aggiornare data e portarlo in uscita affinché venga scritto in memoria quando necessario

Il reset non avviene solamente tramite il segnale `rst` (input del componente) ma anche attraverso `my_rst` (segnale determinato dall'fsm), in particolare basta che uno dei due segnali sia posto a 1 affinché il registro resettì il suo stato

FSM



S0-S1: gestione della la fase iniziale, cioè attesa dello start e inizializzazione

S2-S3-S4-S5-S6: è un ciclo completo di lettura di un numero e scrittura della credibilità, in cui nello stato S2 si decide se proseguire con un nuovo ciclo oppure terminare.

SF-SN: gestione della parte di terminazione della singola esecuzione e reset

Per semplicità vengono definiti i default dei segnali e modificati opportunamente negli stati, i segnali di default sono i seguenti:

- $k_sel = 1$
- $k_load = 0$
- $add_sel = 1$
- $add_load = 0$
- $c_sel = 1$
- $c_reset_sel = 0$
- $c_load = 0$
- $data_load = 0$
- $o_mem_we = 0$

- $o_mem_en = 0$
- $o_mem_addr = o_reg_add$
- $o_mem_data = o_reg_data$
- $o_done = 0$
- $my_rst = 0$

Spiegazione dettagliata stati:

Stato	Output	Descrizione
S0	/	attesa dello start
S1	$k_sel = 0$ $k_load = 1$ $add_sel = 0$ $add_load = 1$ $c_sel = 0$ $c_load = 1$	vengono inizializzati i registri di k e add con i valori letti da i_k e i_add e viene inizializzato il registro di c a 0
S2	$o_mem_en = 1$	viene alzato il segnale per leggere dalla memoria, nello stato successivo o_mem_data conterrà il valore letto da memoria
S3	$c_load = 1$ IF (i_mem_data != 0): $c_reset_sel = 1$ $data_load = 1$	viene scelto se caricare nei registri i valori aggiornati di c e data, in base al valore letto in memoria, questo determinerà poi quali valori verranno scritti in memoria. In particolare se il valore 'w' letto in memoria è 0, il registro di c viene aggiornato (con il suo valore sottratto ad 1) e quello di data non viene aggiornato. Se invece viene letto un valore diverso da 0, il registro di c viene resettato a 31 e quello di data viene aggiornato con il valore appena letto
S4	$o_mem_en = 1$ $o_mem_we = 1$	vengono alzati i segnali per scrivere in memoria, di default il valore scritto in memoria è quello letto dal registro 'data' nella posizione letta dal registro 'add', quindi nel prossimo stato verrà sovrascritto il valore 'w' in memoria; in base allo stato S3 verrà sovrascritto il valore 'w' appena letto o quello salvato precedentemente nel registro
S5	$o_mem_en = 1$ $o_mem_we = 1$ $o_mem_addr = o_reg_add + 1$ $o_mem_data = o_reg_c$	vengono alzati i segnali per scrivere in memoria e vengono modificati i valori di o_mem_data e o_mem_addr affinché venga scritta la credibilità nell'indirizzo successivo ad add
S6	$k_load = 1$ $add_load = 1$	è finito un ciclo di aggiornamento della memoria, vengono aggiornati i valori di k e add nel registro, alzando i segnali di load dei registri
SF	$o_done = 1$	è finita un'esecuzione (sono stati letti k coppie di numeri), viene quindi alzato il segnale o_done
SN	$my_reset = 1$	dopo la fine dell'esecuzione, quando viene abbassato il segnale di start, viene abbassato il segnale o_done e alzato il segnale di reset, che inizializza il registro data

Risultati sperimentali

Sintesi

Timing Report:

```
Max Delay Paths
-----
Slack (MET) : 16.521ns (required time - arrival time)
Source:      FSM_sequential_cur_state_reg[1]/C
              (rising edge-triggered cell FDCE clocked by clock {rise@0.000ns fall@5.000ns period=20.000ns})
Destination: o_reg_add_reg[13]/D
              (rising edge-triggered cell FDCE clocked by clock {rise@0.000ns fall@5.000ns period=20.000ns})
Path Group:  clock
Path Type:   Setup (Max at Slow Process Corner)
Requirement: 20.000ns (clock rise@20.000ns - clock rise@0.000ns)
Data Path Delay: 3.361ns (logic 1.834ns (54.567%) route 1.527ns (45.433%))
Logic Levels: 5 (CARRY4=4 LUT4=1)
Clock Path Skew: -0.145ns (DCD - SCD + CPR)
  Destination Clock Delay (DCD): 2.100ns = ( 22.100 - 20.000 )
  Source Clock Delay (SCD): 2.424ns
  Clock Pessimism Removal (CPR): 0.178ns
Clock Uncertainty: 0.035ns ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
  Total System Jitter (TSJ): 0.071ns
  Total Input Jitter (TIJ): 0.000ns
  Discrete Jitter (DJ): 0.000ns
  Phase Error (PE): 0.000ns
```

L'esecuzione di un ciclo di clock ha una durata di 3.361ns, coerentemente con la richiesta di essere minore di 20ns.

Utilization Report:

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	73	0	134600	0.05
LUT as Logic	73	0	134600	0.05
LUT as Memory	0	0	46200	0.00
Slice Registers	43	0	269200	0.02
Register as Flip Flop	43	0	269200	0.02
Register as Latch	0	0	269200	0.00
F7 Muxes	0	0	67300	0.00
F8 Muxes	0	0	33650	0.00

Non sono stati sintetizzati registri di tipo Latch, ma esclusivamente di tipo Flip Flop.

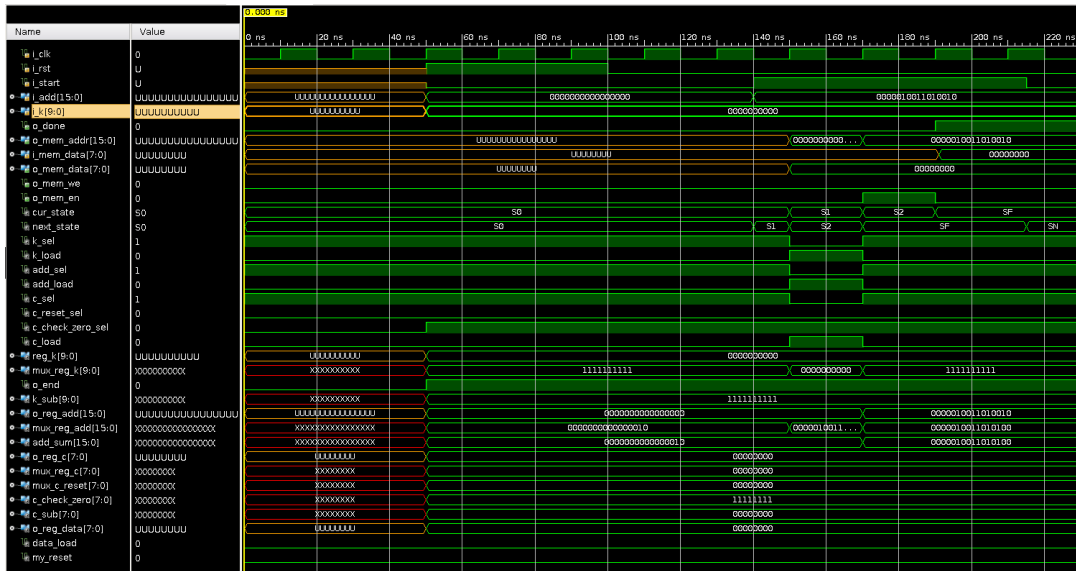
Simulazioni

Test bench 1

Obiettivo: controllare il caso limite in cui la memoria non ha dati e $k = 0$.

Memoria iniziale= []

Memoria dopo esecuzione = []



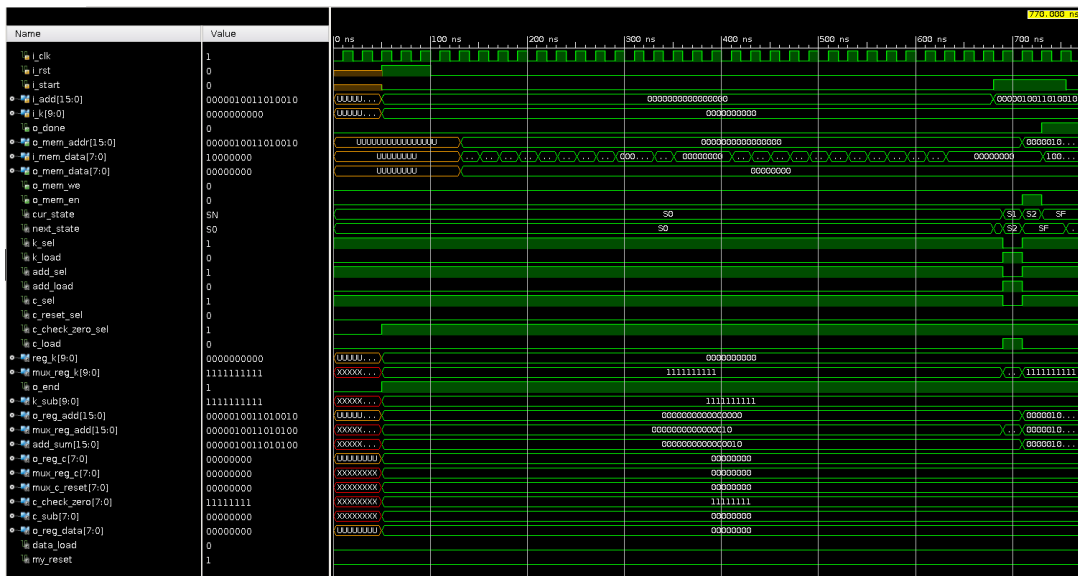
Come da specifica, dopo il segnale di *start*, il componente nello stato S2 alza il segnale *o_end* (poiché $k = 0$) e quindi passa allo stato SF, alzando il segnale *o_done*.

Test bench 2

Obiettivo: controllare il caso limite in cui la memoria non è vuota ma $k = 0$.

scenario 1 = [128, 0, 64, 23, 0, 12, 0, 71, 0, 0, 14, 0, 0, 0, 100, 0, 1, 0, 91, 0, 5, 0, 23, 0, 200, 0, 0, 0]

scenario 1 dopo esecuzione = [128, 0, 64, 23, 0, 12, 0, 71, 0, 0, 14, 0, 0, 0, 100, 0, 1, 0, 91, 0, 5, 0, 23, 0, 200, 0, 0, 0]



Come da specifica, indipendentemente dalla memoria, poiché $k = 0$, si ha lo stesso comportamento del test bench 1.

Test bench 3

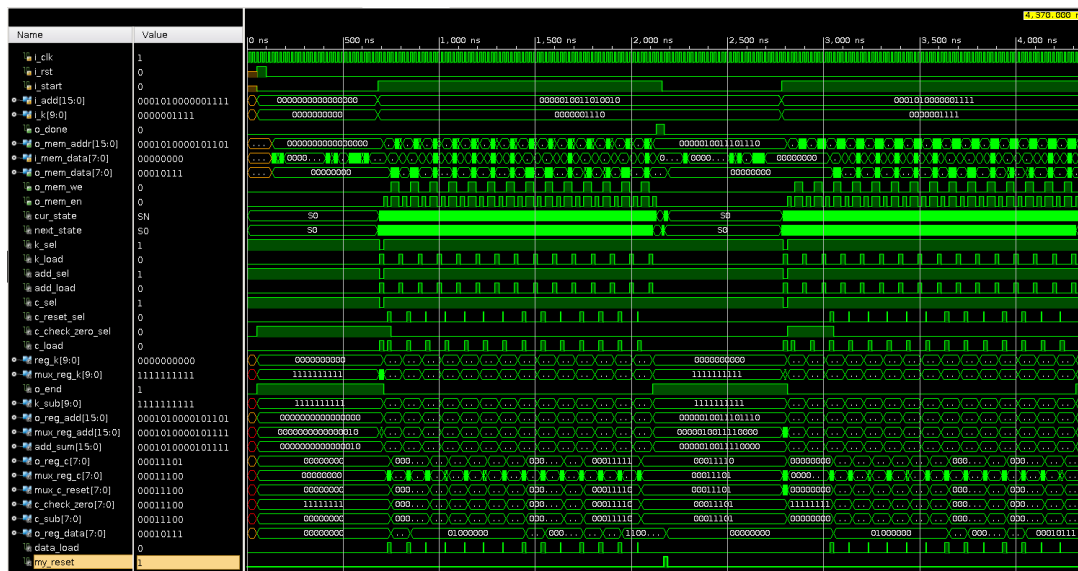
Obiettivo: controllare il caso limite in cui ci sono più start, nel caso specifico 2 start.

scenario 1 = [128, 0, 64, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 100, 0, 1, 0, 0, 0, 5, 0, 23, 0, 200, 0, 0, 0]

scenario 1 dopo esecuzione = [128, 31, 64, 31, 64, 30, 64, 29, 64, 28, 64, 27, 64, 26, 100, 31, 1, 31, 1, 30, 5, 31, 23, 31, 200, 31, 200, 30]

scenario 2 = [0, 0, 0, 0, 64, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 150, 0, 1, 0, 0, 0, 5, 0, 23, 0, 0, 0, 0]

scenario 2 dopo esecuzione = [0, 0, 0, 0, 64, 31, 64, 30, 64, 29, 64, 28, 64, 27, 64, 26, 150, 31, 1, 31, 1, 30, 5, 31, 23, 31, 23, 30, 23, 29]



Come da specifica, una volta conclusa la prima esecuzione, viene portato il segnale *o_done* a 1 e successivamente portato a 0 una volta abbassato il segnale di *start*.

Successivamente il componente resetta il registro *reg_data* e rimane nello stato S0 fino a quando non riceve il segnale di *start*, una volta ricevuto, processa la seconda richiesta di codifica correttamente

Il reset di *reg_data* una volta finita un'esecuzione viene reso necessario dalla possibilità che lo scenario successivo abbia uno 0 come primo numero letto, come accade in questo caso di test.

Test bench 4

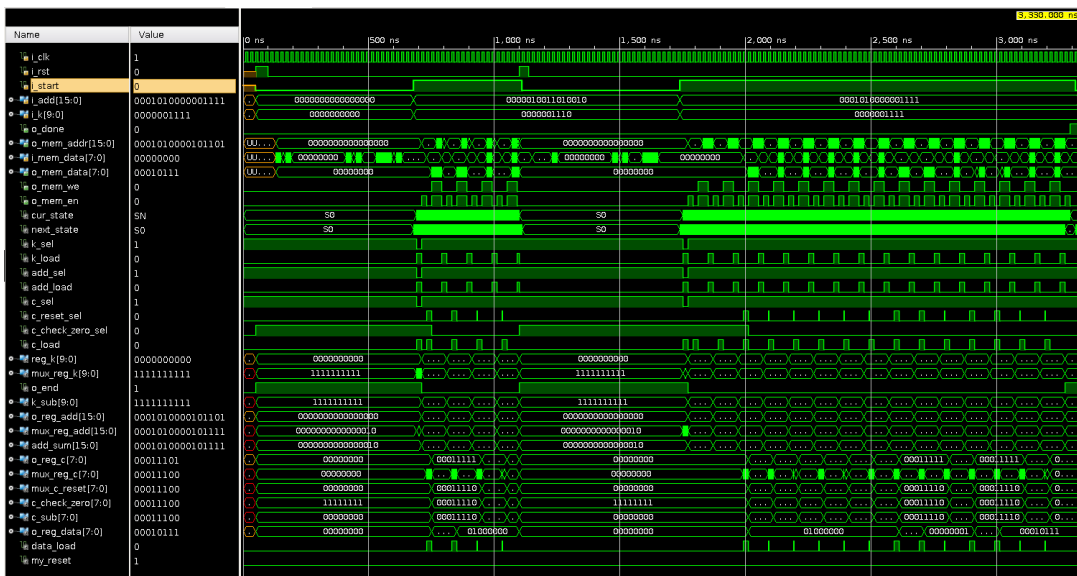
Obiettivo: controlla il caso limite in cui c'è un reset durante l'esecuzione.

scenario 1 = [128 0, 64, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 100, 0, 1, 0, 0, 0, 5, 0, 23, 0, 200, 0, 0, 0]

scenario 1 dopo esecuzione = [128, 31, 64, 31, 64, 30, 64, 29, 64, 28, 64, 27, 64, 26, 100, 31, 1, 31, 1, 30, 5, 31, 23, 31, 200, 31, 200, 30]

scenario 2 = [0, 0, 0, 0, 64, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 150, 0, 1, 0, 0, 0, 5, 0, 23, 0, 0, 0, 0, 0]

scenario 2 dopo esecuzione = [0, 0, 0, 0, 64, 31, 64, 30, 64, 29, 64, 28, 64, 27, 64, 26, 150, 31, 1, 31, 1, 30, 5, 31, 23, 31, 23, 30, 23, 29]



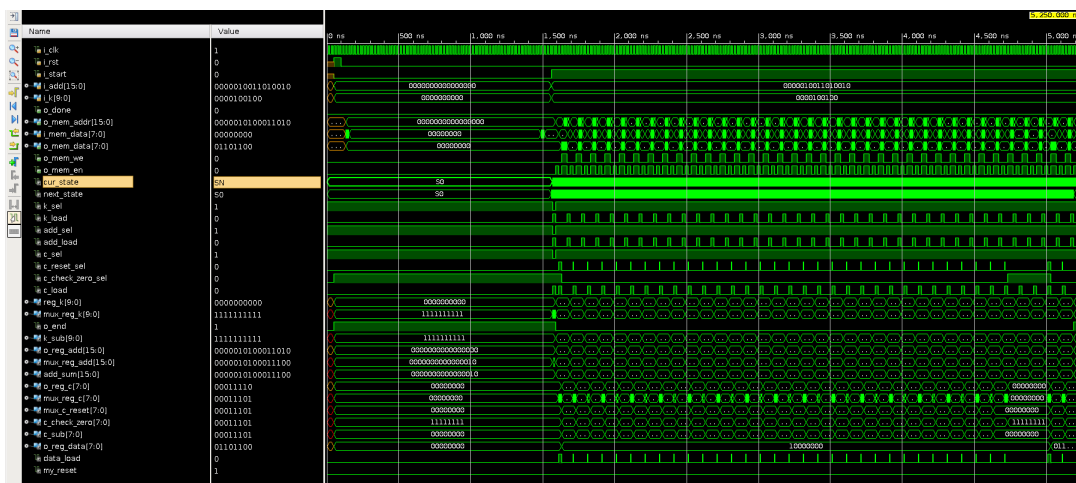
Come da specifica, il componente, una volta ricevuto il segnale di *reset*, resetta tutti i registri e torna nello stato S0, una volta ricevuto un nuovo segnale di *start* processa la nuova richiesta di codifica correttamente.

Test bench 5

Obiettivo: controllare il caso limite in cui la credibilità scende fino a 0, in particolare viene testato il funzionamento del controllo di c non negativo del modulo 2.

[illegible]

scenario 1 dopo esecuzione = [128, 31, 128, 30, 128, 29, 128, 28, 128, 27, 128, 26, 128, 25, 128, 24, 128, 23, 128, 22, 128, 21, 128, 20, 128, 19, 128, 18, 128, 17, 128, 16, 128, 15, 128, 14, 128, 13, 128, 12, 128, 11, 128, 10, 128, 9, 128, 8, 128, 7, 128, 6, 128, 5, 128, 4, 128, 3, 128, 2, 128, 1, 128, 0, 128, 0, 128, 0, 108, 31, 108, 30]



Come da specifica, la credibilità, una volta raggiunto il valore 0, rimane tale e non scende sotto zero, successivamente, una volta che viene letto un numero diverso da zero, viene resettata correttamente a 31.

Conclusioni

Il componente è stato testato non esclusivamente sui test bench commentati, ma su un elevato numero di essi, i 5 commentati sono di maggior valenza esplicativa in quanto valutano specifici casi limite. Il componente supera correttamente tutti i test bench sia in modalità behavioral che post synthesis.

Il comportamento in modalità behavioral è analogo a quello del componente sintetizzato, anche grazie all'assenza di un involontaria introduzione di registri di tipo latch.

Inoltre i vincoli di clock vengono ampiamente rispettati, con un'esecuzione massima per ciclo di clock di 3.361ns, ben al di sotto del vincolo di 20ns.

Per quanto riguarda le scelte progettuali ho utilizzato come FSM una macchina di Mealy.

Sarebbe stato possibile dividere lo stato S3 in due stati, uno raggiunto se *i_mem_data* = 0, l'altro se *i_mem_data* != 0, affinché le uscite in ognuno di questi due stati dipendesse soltanto dallo stato e non dagli ingressi, come per ogni altro stato. Questo avrebbe reso l'FSM una macchina di Moore.

Tuttavia ho preferito utilizzare uno stato in meno piuttosto che utilizzare una macchina di Moore.